

Aritmetica dei Calcolatori

Aritmetica dei Calcolatori

- I calcolatori eseguono somme e sottrazioni su base binaria nel seguente modo:
 - Prima viene eseguita la somma delle cifre più a destra (chiamate anche LSB: Less Significant Bit)
 - Successivamente viene calcolato il riporto
 - Alla prossima somma vengono sommati i bit dei numeri più il riporto

0	0	0	1	(carry)	1	1	1	1
	1	0	0	x		1	0	1
+	0	1	0	y	+	0	1	1
0	1	1	1	$x + y$	1	0	0	1
no overflow					overflow			

- Se la somma finale ha il riporto si dice che abbiamo overflow, altrimenti non c'è overflow

Aritmetica dei Calcolatori

- Dati n bit è possibile rappresentare 2^n numeri. Chiamiamo tale insieme di combinazioni **Spazio dei Numeri**
- Per rappresentare i numeri negativi una soluzione naturale è dividere in due lo Spazio dei Numeri e usarne una metà per rappresentare i numeri negativi.
- Il metodo utilizzato dai moderni computer viene chiamato complemento a 2.
 - In complemento a 2 tutti i numeri negativi hanno il bit più a sinistra a 1
 - In complemento a 2 tutti i numeri positivi hanno il bit più a sinistra a 0

Aritmetica dei Calcolatori

Positive numbers		Negative numbers	
0	0000		
1	0001	1111	−1
2	0010	1110	−2
3	0011	1101	−3
4	0100	1100	−4
5	0101	1011	−5
6	0110	1010	−6
7	0111	1001	−7
		1000	−8

Aritmetica dei Calcolatori

- Cambiare di segno un numero in complemento a due:
 - Copio tutti i bit fino al primo 1, gli altri li copio cambiati di segno
 - Esempio:
 - $(-3) \ 1101 \rightarrow (3) \ 0011$
 - $(7) \ 0111 \rightarrow (-7) \ 1001$
- Il complemento a due è utile per eseguire la sottrazione in quanto basta cambiare di segno un numero ed eseguire una normale somma.

Aritmetica dei Calcolatori

La caratteristica interessante di questa rappresentazione dei numeri negativi è che la somma di due numeri con segno non cambia rispetto alla somma di due numeri normali.

Consideriamo la somma di $(-2)+(-3)$. Convertiamo entrambe i numeri in binario, 4 bit, complemento a 2 e si ottiene: $1110 + 1101$

```
1110+
1101=
-----
1011 → (-5)
```

Aritmetica dei Calcolatori

- Esempi (rappresentazione dei numeri con 4 bit):

- Sommare i numeri 2 e 3:

0010+

0011=

0101 (5)

- Sottrarre i numeri 2 e 3:

- Trasformo il 3 in binario 0011

- Lo faccio diventare un -3 con complemento a 2: 1101

- Eseguo la somma in colonna:

0010+

1101=

1111 (-1)

Aritmetica dei Calcolatori

- Il complemento a 2 oltre che facilitare la somma di numeri con segno consente anche di effettuare sottrazioni.
- Per effettuare una sottrazione basta cambiare di segno il secondo numero ed effettuare una normale somma.

Aritmetica dei Calcolatori

- La parte della CPU che svolge le operazioni aritmetiche viene chiamata ALU (Arithmetic Logic Unit).
- Al suo interno è formata da circuiti sommatore e segnali di controllo.
- I circuiti sommatore principali sono:
 - Half-adder: circuito sommatore ad un bit
 - Full-adder: circuito sommatore a due bit
 - Adder: circuito sommatore a n bit
 - Incrementer: circuito che incrementa di 1 il numero in ingresso

Circuiti sommatori

- Con le porte logiche a disposizione è possibile creare circuiti sommatore (che sommano bit tra loro).
- 3 circuiti sommatore fondamentali:
 - Half Adder: somma due bit
 - Full Adder: somma tre bit
 - Adder: somma n bit
 - Incrementer: incrementa di 1 un numero di n bit

Circuiti sommatore

- Creare un Half Adder con le seguenti caratteristiche

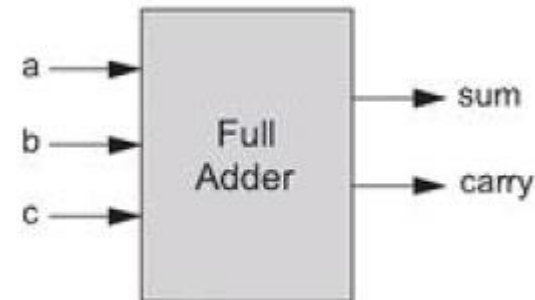
Inputs		Outputs	
a	b	carry	sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Circuiti sommatori

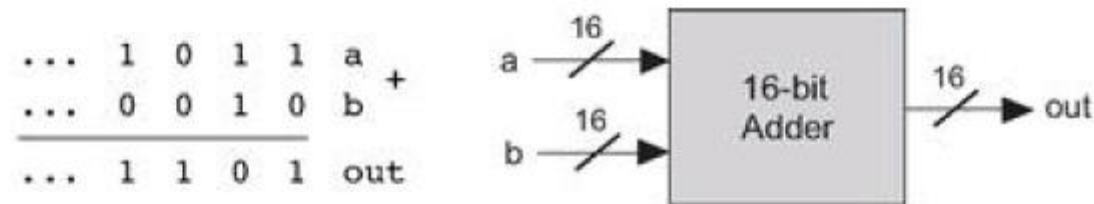
- Creare un Full Adder con le seguenti caratteristiche

a	b	c	carry	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Circuiti sommatori

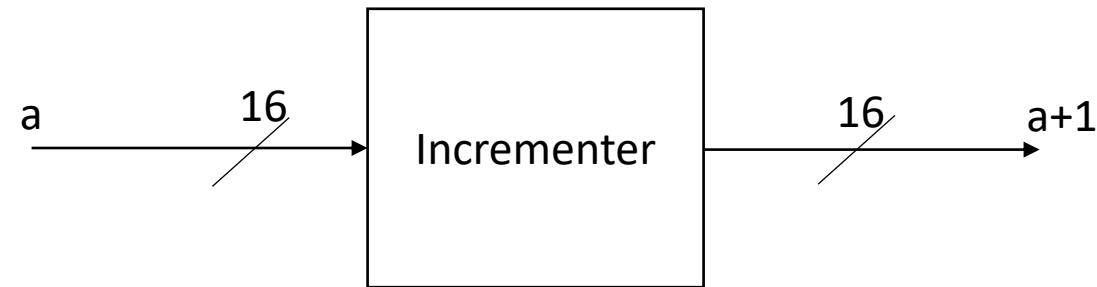
- Creare un Adder con le seguenti caratteristiche



- L'uscita di riporto può essere omessa

Circuiti sommatori

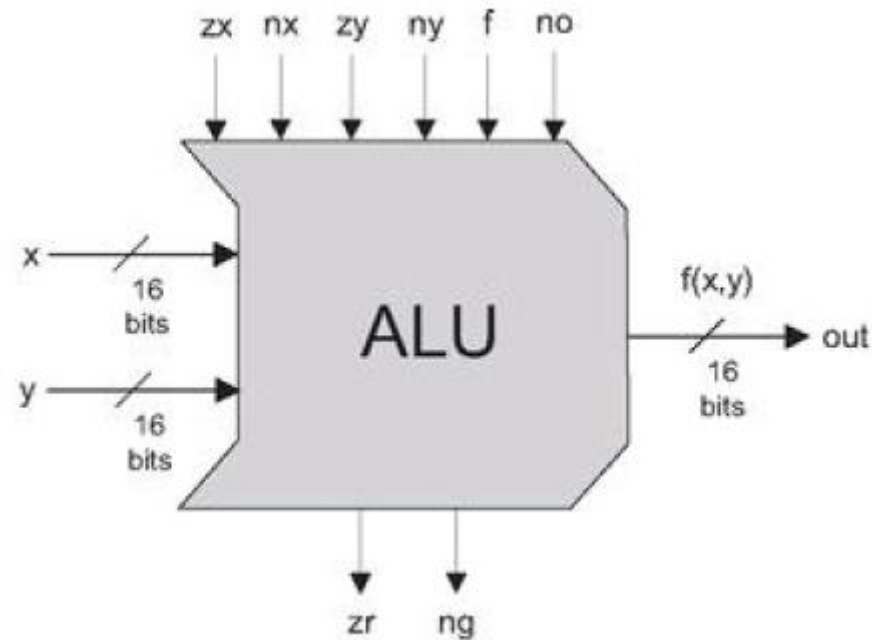
- Creare un Incrementer con le seguenti caratteristiche



- L'uscita di riporto può essere omessa

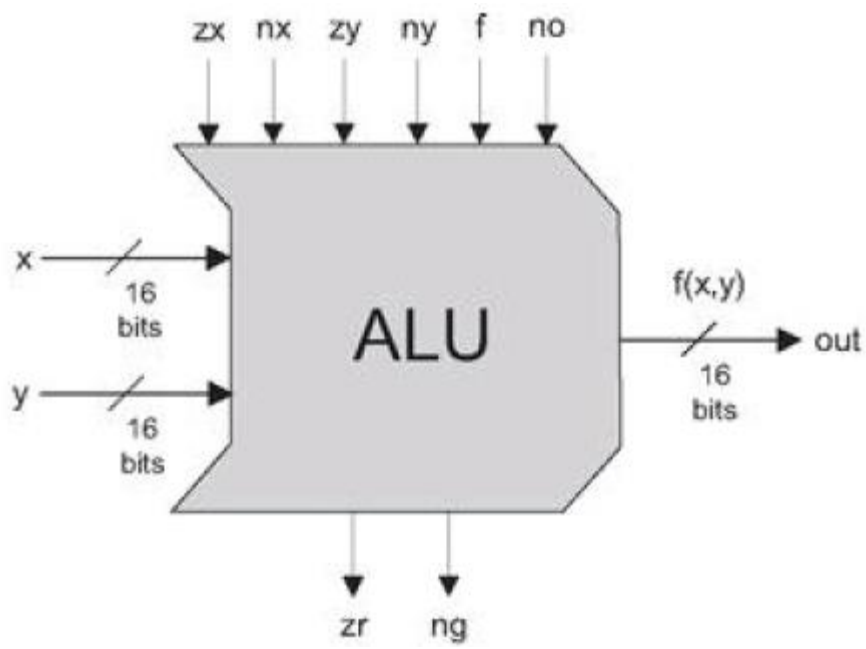
Circuiti sommatori

- Creare una ALU con le seguenti caratteristiche:



```
Chip name: AIX
Inputs:  x(16), y(16), // Two 16-bit data inputs
        zx,           // Zero the x input
        nx,           // Negate the x input
        zy,           // Zero the y input
        ny,           // Negate the y input
        f,            // Function code: 1 for Add, 0 for And
        no            // Negate the out output
Outputs: out(16),      // 16-bit output
        zr,           // True iff out=0
        ng            // True iff out<0
Function: if zx then x = 0 // 16-bit zero constant
        if nx then x = !x // Bit-wise negation
        if zy then y = 0 // 16-bit zero constant
        if ny then y = !y // Bit-wise negation
        if f then out = x + y // Integer 2's complement addition
            else out = x & y // Bit-wise And
        if no then out = !out // Bit-wise negation
        if out=0 then zr = 1 else zr = 0 // 16-bit eq. comparison
        if out<0 then ng = 1 else ng = 0 // 16-bit neg. comparison
Comment: Overflow is neither detected nor handled.
```


These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y



These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y